ARVIND SCHOOL MANAGER

A second long Python desktop-app project. This PDF contains the complete source code for a school manager with a dashboard, student registration, searchable student records, announcements, and local JSON saving.

How to run: Save the code as `arvind_school_manager.py`. Run it on a computer with Python 3 and Tkinter installed. The app creates `school_data.json` in the same folder to store records.

Complete Python Code

```
import tkinter as tk
from tkinter import messagebox, ttk
import json
import os
from datetime import datetime

# ============================================================
# ARVIND SCHOOL MANAGER
# A desktop school-management app made with Python + Tkinter
# ============================================================

APP_TITLE = "ARVIND SCHOOL MANAGER"
DATA_FILE = "school_data.json"

# ---------- DATA ----------
def load_data():
    if os.path.exists(DATA_FILE):
        try:
            with open(DATA_FILE, "r", encoding="utf-8") as file:
                saved = json.load(file)
                return {
                    "students": saved.get("students", []),
                    "announcements": saved.get("announcements", [])
                }
        except (OSError, json.JSONDecodeError):
            pass
    return {"students": [], "announcements": []}


data = load_data()


def save_data():
    with open(DATA_FILE, "w", encoding="utf-8") as file:
        json.dump(data, file, indent=4)


# ---------- MAIN WINDOW ----------
root = tk.Tk()
root.title(APP_TITLE)
root.geometry("900x650")
root.minsize(760, 550)
root.configure(bg="#eef2f7")


def clear_content():
    for widget in content.winfo_children():
        widget.destroy()


def make_button(parent, label, command, color="#2457a7"):
    return tk.Button(
        parent, text=label, command=command,
        bg=color, fg="white", activebackground="#173d78",
        activeforeground="white", font=("Arial", 10, "bold"),
        relief="flat", padx=12, pady=8, cursor="hand2"
    )


def page_title(title, subtitle=""):
    tk.Label(
        content, text=title, bg="#eef2f7", fg="#172b4d",
        font=("Arial", 23, "bold")
    ).pack(anchor="w", padx=22, pady=(20, 4))
    if subtitle:
        tk.Label(
            content, text=subtitle, bg="#eef2f7", fg="#596579",
            font=("Arial", 10)
        ).pack(anchor="w", padx=24, pady=(0, 15))
# ---------- TOP BAR AND NAVIGATION ----------
topbar = tk.Frame(root, bg="#172b4d", height=65)
```

```
topbar.pack(fill="x")
topbar.pack_propagate(False)

tk.Label(
    topbar, text="■  ARVIND SCHOOL MANAGER",
    bg="#172b4d", fg="white", font=("Arial", 18, "bold")
).pack(side="left", padx=20)

content = tk.Frame(root, bg="#eef2f7")
content.pack(fill="both", expand=True)


# ---------- STUDENT HELPERS ----------
def get_student_by_id(student_id):
    for student in data["students"]:
        if student["id"].lower() == student_id.lower():
            return student
    return None


def refresh_student_table(tree, students=None):
    for item in tree.get_children():
        tree.delete(item)

    source = data["students"] if students is None else students
    for student in source:
        tree.insert(
            "", "end",
            values=(
                student["id"], student["name"], student["class"],
                student["age"], student["phone"]
            )
        )


# ---------- DASHBOARD ----------
def show_dashboard():
    clear_content()
    page_title("Dashboard", "A quick overview of school records.")

    total = len(data["students"])
    classes = len(set(s["class"] for s in data["students"]))

    cards = tk.Frame(content, bg="#eef2f7")
    cards.pack(fill="x", padx=20, pady=15)

    for label, value, color in [
        ("TOTAL STUDENTS", total, "#2457a7"),
        ("CLASSES", classes, "#16836b"),
        ("ANNOUNCEMENTS", len(data["announcements"]), "#9a5a13")
    ]:
        card = tk.Frame(cards, bg="white", padx=22, pady=18,
                        highlightbackground="#d6deea", highlightthickness=1)
        card.pack(side="left", fill="both", expand=True, padx=7)
        tk.Label(card, text=str(value), bg="white", fg=color,
                 font=("Arial", 27, "bold")).pack()
        tk.Label(card, text=label, bg="white", fg="#596579",
                 font=("Arial", 9, "bold")).pack(pady=(5, 0))

    tk.Label(
        content, text="Recent students", bg="#eef2f7",
        fg="#172b4d", font=("Arial", 15, "bold")
    ).pack(anchor="w", padx=25, pady=(20, 8))

    tree = ttk.Treeview(
        content, columns=("id", "name", "class", "age", "phone"),
        show="headings", height=8
    )
    for col, width in [
        ("id", 100), ("name", 210), ("class", 120),
        ("age", 70), ("phone", 150)
    ]:
        tree.heading(col, text=col.upper())
        tree.column(col, width=width, anchor="center")
    tree.pack(fill="x", padx=25)
    refresh_student_table(tree)


# ---------- ADD STUDENT ----------
def add_student_page():
    clear_content()
    page_title("Add Student", "Enter the student's details and save the record.")

    form = tk.Frame(content, bg="white", padx=25, pady=22)
    form.pack(anchor="nw", padx=25, pady=10)

    fields = {}
    labels = ["Student ID", "Full name", "Class / Grade", "Age", "Phone"]
    for row, label in enumerate(labels):
        tk.Label(form, text=label, bg="white", fg="#26364d",
                 font=("Arial", 10, "bold")).grid(
                     row=row, column=0, sticky="w", pady=9, padx=(0, 20)
```

```
                )
        entry = tk.Entry(form, width=34, font=("Arial", 11))
        entry.grid(row=row, column=1, pady=9)
        fields[label] = entry

    def save_student():
        values = {key: entry.get().strip() for key, entry in fields.items()}
        if any(not value for value in values.values()):
            messagebox.showwarning("Missing information", "Please fill in every field.")
            return
        if get_student_by_id(values["Student ID"]):
            messagebox.showerror("Duplicate ID", "That student ID already exists.")
            return
        if not values["Age"].isdigit():
            messagebox.showerror("Invalid age", "Age must be a whole number.")
            return

        data["students"].append({
            "id": values["Student ID"],
            "name": values["Full name"],
            "class": values["Class / Grade"],
            "age": values["Age"],
            "phone": values["Phone"],
            "created": datetime.now().strftime("%Y-%m-%d %H:%M")
        })
        save_data()
        messagebox.showinfo("Saved", "Student record added successfully.")
        show_dashboard()

    make_button(form, "SAVE STUDENT", save_student).grid(
        row=len(labels), column=1, sticky="e", pady=(18, 0)
    )


# ---------- VIEW AND SEARCH STUDENTS ----------
def student_list_page():
    clear_content()
    page_title("Student Records", "Search, view, and remove student records.")

    search_bar = tk.Frame(content, bg="#eef2f7")
    search_bar.pack(fill="x", padx=25, pady=5)

    search_entry = tk.Entry(search_bar, width=35, font=("Arial", 11))
    search_entry.pack(side="left", padx=(0, 8), ipady=5)

    columns = ("id", "name", "class", "age", "phone")
    tree = ttk.Treeview(content, columns=columns, show="headings", height=14)
    widths = {"id": 100, "name": 220, "class": 130, "age": 70, "phone": 160}
    for col in columns:
        tree.heading(col, text=col.upper())
        tree.column(col, width=widths[col], anchor="center")
    tree.pack(fill="both", expand=True, padx=25, pady=12)

    def search():
        query = search_entry.get().strip().lower()
        matches = [
            s for s in data["students"]
            if query in s["id"].lower()
            or query in s["name"].lower()
            or query in s["class"].lower()
        ]
        refresh_student_table(tree, matches)

    def delete_selected():
        selected = tree.selection()
        if not selected:
            messagebox.showwarning("Select a student", "Choose a row first.")
            return
        student_id = tree.item(selected[0], "values")[0]
        if messagebox.askyesno("Confirm delete", f"Delete student {student_id}?"):
            data["students"] = [
                s for s in data["students"] if s["id"] != student_id
            ]
            save_data()
            refresh_student_table(tree)

    make_button(search_bar, "SEARCH", search).pack(side="left")
    make_button(search_bar, "SHOW ALL", lambda: refresh_student_table(tree),
                "#596579").pack(side="left", padx=7)
    make_button(content, "DELETE SELECTED STUDENT", delete_selected,
                "#b33a3a").pack(anchor="e", padx=25, pady=(0, 15))

    refresh_student_table(tree)


# ---------- ANNOUNCEMENTS ----------
def announcements_page():
    clear_content()
    page_title("Announcements", "Write and save a message for the school notice board.")

    editor = tk.Text(content, height=5, font=("Arial", 11), wrap="word")
    editor.pack(fill="x", padx=25, pady=10)
```

```
    def add_announcement():
        message = editor.get("1.0", "end").strip()
        if not message:
            messagebox.showwarning("Empty message", "Write an announcement first.")
            return
        data["announcements"].insert(0, {
            "message": message,
            "date": datetime.now().strftime("%d %b %Y, %I:%M %p")
        })
        save_data()
        messagebox.showinfo("Posted", "Announcement saved.")
        announcements_page()

    make_button(content, "POST ANNOUNCEMENT", add_announcement).pack(
        anchor="w", padx=25, pady=5
    )

    tk.Label(content, text="Saved announcements", bg="#eef2f7",
             fg="#172b4d", font=("Arial", 14, "bold")).pack(
                 anchor="w", padx=25, pady=(18, 5)
             )

    board = tk.Frame(content, bg="#eef2f7")
    board.pack(fill="both", expand=True, padx=25, pady=5)

    for item in data["announcements"]:
        box = tk.Frame(board, bg="white", padx=12, pady=10)
        box.pack(fill="x", pady=5)
        tk.Label(box, text=item["message"], bg="white", fg="#26364d",
                 wraplength=760, justify="left").pack(anchor="w")
        tk.Label(box, text=item["date"], bg="white", fg="#788397",
                 font=("Arial", 8)).pack(anchor="w", pady=(5, 0))


# ---------- ABOUT PAGE ----------
def about_page():
    clear_content()
    page_title("About This App")
    tk.Label(
        content,
        text=(
            "ARVIND SCHOOL MANAGER\n\n"
            "Version: 1.0\n"
            "Built using Python and Tkinter\n\n"
            "Features:\n"
            "• Student registration\n"
            "• Student record search\n"
            "• Delete student records\n"
            "• School announcements\n"
            "• Dashboard summary\n\n"
            "Records are saved locally in school_data.json."
        ),
        bg="white", fg="#26364d", justify="left",
        font=("Arial", 12), padx=25, pady=22
    ).pack(anchor="nw", padx=25, pady=10)


# ---------- LEFT SIDEBAR ----------
sidebar = tk.Frame(root, bg="#243b5a", width=185)
sidebar.pack(side="left", fill="y", before=content)
sidebar.pack_propagate(False)

tk.Label(
    sidebar, text="MAIN MENU", bg="#243b5a", fg="#b9c9df",
    font=("Arial", 10, "bold")
).pack(anchor="w", padx=17, pady=(22, 12))

def nav_button(label, command):
    tk.Button(
        sidebar, text=label, command=command,
        bg="#243b5a", fg="white", activebackground="#36577f",
        activeforeground="white", relief="flat",
        font=("Arial", 10), anchor="w", padx=17, pady=11
    ).pack(fill="x")

nav_button("Dashboard", show_dashboard)
nav_button("Add Student", add_student_page)
nav_button("Student Records", student_list_page)
nav_button("Announcements", announcements_page)
nav_button("About", about_page)

tk.Label(
    sidebar, text="Local desktop app",
    bg="#243b5a", fg="#b9c9df", font=("Arial", 8)
).pack(side="bottom", pady=18)

# Start on dashboard
show_dashboard()
root.mainloop()
```